Image Steganography Report

1. Capacity Calculations

Formula:

- pixels = width × height
- capacity_bits = pixels
- capacity_bytes = capacity_bits / 8
- message_bits = 14 + 7 × number_of_characters
- complete_copies = capacity_bits // message_bits

Images:

1. dice.bmp

- Dimensions: 400 × 454
- Pixels: 400 × 454 = **181600**
- Capacity:
  - Bits: **181600 bits**
  - Bytes: **22700 bytes**

---

2. flowers.bmp

- Dimensions: 500 × 300
- Pixels: 500 × 300 = **150000**
- Capacity:
  - Bits: **150000 bits**
  - Bytes: **18750 bytes**

---

3. tiger.bmp

- Dimensions: 520 × 330
- Pixels: 520 × 330 = **171600**
- Capacity:
  - Bits: **171600 bits**
  - Bytes: **21450 bytes**

Secret Message

- Number of characters: **42**
- Message bits:
  - 14 + 7 × 42 = **308 bits**

Complete Copies

Image	Capacity (bits)	Copies
dice.bmp	181600	589
flowers.bmp	150000	487
tiger.bmp	171600	557

Conclusion

The image **dice.bmp** can store the largest hidden message because it has the highest number of pixels.
File size alone is not the best measure of capacity; the number of pixels determines how much data can be hidden.

---

2. Experiments

Colour Plane Changes

- **Red channel (0):** Slight visual changes
- **Green channel (1):** Least noticeable changes (best choice)
- **Blue channel (2):** Slight changes, sometimes visible

Bit Position Changes

- **bit_position = 7 (LSB):**
  Minimal visual distortion, almost invisible
- **bit_position = 6 or 5:**
  Slight visible distortion begins
- **bit_position = 0 (MSB):**
  Strong distortion, image visibly corrupted

Different Images

- Larger images (more pixels) have higher capacity
- Smaller images fill faster and may distort earlier

Wrong Key Test

When using an incorrect key:

- The extracted message becomes random and unreadable
- This happens because pixel order is different

---

3. Short Explanations

1. Why is the hidden message difficult to notice?

Because the algorithm modifies only the least significant bit (LSB), which causes very small changes in pixel values that are not visible to the human eye.

---

2. Which parameter is most important?

The **key** is the most important parameter because it determines the pixel order. Without the correct key, extraction fails.

---

3. Why does extraction fail with wrong key?

Because pixels are read in the wrong order, so the reconstructed bits do not form the original message.

---

4. Two weaknesses of this system

- Not resistant to image compression (e.g. JPG)
- Easy to break if the method is known

---

5. How could the system be improved?

- Encrypt the message before embedding
- Use multiple bits or adaptive embedding
- Support images or logos instead of text

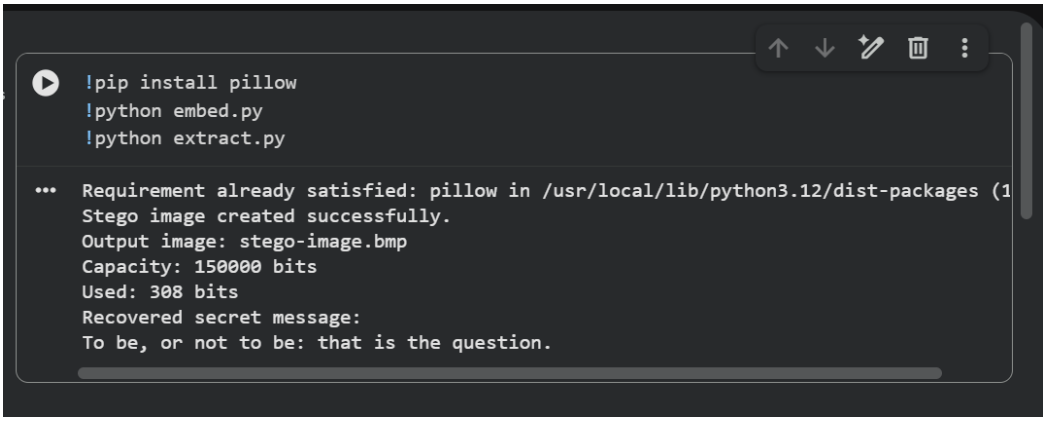
---

4. Reflection

This steganography method is effective because it hides data in a way that is not visually detectable. However, it is not secure against advanced attacks or image modifications. The use of a key improves security but does not make the system fully robust.

---

5. Screenshot



`python extract.py`
